

Exception-Handling – Teil 1

try, catch und finally

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 19. Januar 2026 }

inno



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Exception-Handling

 Plan für diese Woche

 Houston, wir haben eine Exception

 Geprüfte und Ungeprüfte

 Flusskontrolle

 finally-Block

 Pair Programming

 Spaß

 Mehrere Exceptions

 Polymorphe Exceptions

 Ausweichen

 Exception-Regeln

Exception-Handling

Verbindungen herstellen

 Tauschen Sie sich mit einer Nachbar:in aus!

 Wozu dienen Exceptions?

 Was ist der Unterschied zwischen geprüften und ungeprüften Ausnahmen?

 Welche Auswirkungen hat das Auftreten einer Ausnahme auf den Kontrollfluss?

 Wann wird ein `finally`-Block ausgeführt?

 Welche Fragen haben Sie zu diesem Kapitel?

13 Exception-Handling

Riskantes Verhalten



Dinge passieren. Eine Datei fehlt. Der Server ist down. Egal wie gut Sie als Programmierer sind, Sie können nicht alles unter Kontrolle haben. Dinge können schiefgehen. *Richtig* schief. Wenn Sie eine riskante Methode schreiben, brauchen Sie Code, der mit den schlimmsten Dingen umgeht, die passieren könnten. Aber woher wissen Sie, *wann* eine Methode riskant ist? Und wo tun Sie den Code hin, der diese **Ausnahmesituation** *behandelt*? Bisher sind wir in diesem Buch keine *echten* Risiken eingegangen. Klar, es sind zur Laufzeit ein paar Dinge danebengegangen, aber der Grund waren meistens Fehler in unserem eigenen Code. Programmierfehler, *Bugs* eben. Und die sollten wir während der Entwicklungsphase reparieren. Nein, hier geht es um Code, für den Sie *nicht* garantieren können, dass er zur Laufzeit funktioniert. Code, der erwartet, dass eine Datei im richtigen Verzeichnis liegt, dass der Server läuft oder dass der Thread nicht aufwacht. Und wir müssen uns *jetzt* darum kümmern. Denn in *diesem* Kapitel bauen wir etwas, das die riskante JavaSound-API verwendet. Wir programmieren ein Programm zum Abspielen von MIDI-Dateien.

Hier fängt ein neues Kapitel an 421

 **Houston, wir haben eine
Exception**

Wofür, wie, was?

🔥 Houston, wir haben eine Exception

Wofür gibt es in Java Exceptions?

- Exceptions erhöhen die Robustheit und Wartbarkeit von Software. Mit Hilfe von Exceptions können Fehler und andere Ausnahmesituationen **kontrolliert** behandelt werden.
- Durch die Verwendung von Exceptions können Sie Ihr Programm auf unerwartete Situationen (z. B. Benutzerfehler,

Systemausfälle, Netzwerkprobleme) **vorbereiten**, ohne dass das Programm abstürzt oder falsche Ergebnisse ausgibt.

- Ein typisches Beispiel wäre, wenn ein Programm eine Datei öffnen soll, die nicht existiert. Anstatt abzustürzen, kann das Programm eine Fehlermeldung ausgeben oder eine Alternative ausprobieren.

Wofür, wie, was?

🔥 Houston, wir haben eine Exception

Wie funktionieren Exceptions?

- Exceptions dienen dazu, den normalen Ablauf eines Programms von der Fehlerbehandlung zu **trennen**.
- Wenn ein Fehler auftritt, wird eine Exception ausgelöst. Dies **unterbricht** den normalen Programmablauf und übergibt die Kontrolle an einen Teil des

Programms, den sogenannten Exception-Handler. Dieser ist für die Behandlung dieser speziellen Fehlerart zuständig.

- Da die Fehlerbehandlungslogik von der Hauptlogik des Programms getrennt bleibt, wird der Code **sauberer** und **einfacher zu warten**.

Wofür, wie, was?

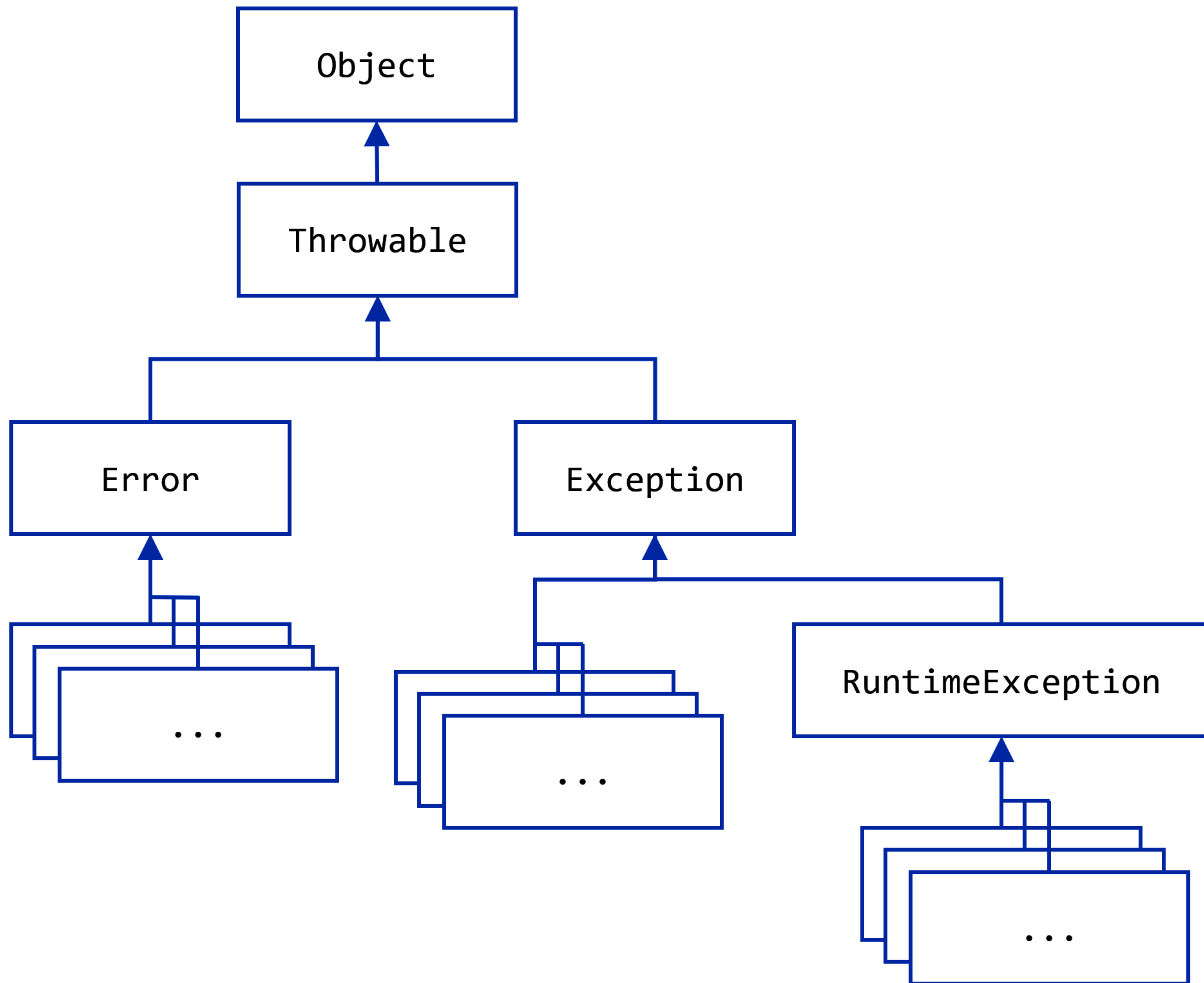
🔥 Houston, wir haben eine Exception

Was sind Exceptions?

- Exceptions sind **Objekte**, die Fehler oder unerwartete Zustände darstellen, die während der Ausführung eines Programms auftreten.
- Es gibt **zwei Arten** von Ausnahmen: **Geprüfte (checked)** Ausnahmen, die explizit abgefangen oder in der

Methodensignatur deklariert werden müssen, und **ungeprüfte (unchecked)** Ausnahmen, die nicht explizit behandelt werden müssen.

- Java bietet eine umfangreiche **Hierarchie** von Ausnahmeklassen. Diese erlauben es, verschiedene Fehlertypen spezifisch und angemessen zu behandeln.



LinkedListException

```
graph BT; InvalidIndexException --> LinkedListException; ObjectNotFoundException --> LinkedListException; EmptyListException --> ObjectNotFoundException;
```

InvalidIndexException

ObjectNotFoundException

EmptyListException

Exceptions

The Java programming language uses *exceptions* to handle errors and other exceptional events. This tutorial describes when and how to use exceptions.

1. [What Is an Exception?](#)

Introducing what exceptions are.

2. [Catching and Handling Exceptions](#)

How to use try, catch and finally.

3. [Throwing Exceptions](#)

How to throw exceptions in your programs.

4. [Unchecked Exceptions — The Controversy](#)

When to Use Checked or Unchecked Exceptions

Module `java.base`
Package `java.nio.file`

Class Files

`java.lang.Object`
`java.nio.file.Files`

`public final class Files`
`extends Object`

This class consists exclusively of static methods that operate on files, directories, or other types of files.

In most cases, the methods defined here will delegate to the associated file system provider to perform the file operations.

Since:
1.7

Method Summary

All Methods	Static Methods	Concrete Methods
Modifier and Type	Method	Description
static long	<code>copy(InputStream in, Path target, CopyOption... options)</code>	Copies all bytes from an input stream to a file.
static long	<code>copy(Path source, OutputStream out)</code>	Copies all bytes from a file to an output

readAllLines

```
public static List<String> readAllLines(Path path)
    throws IOException
```

Read all lines from a file. Bytes from the file are decoded into characters using the UTF-8 charset.

This method works as if invoking it were equivalent to evaluating the expression:

```
Files.readAllLines(path, StandardCharsets.UTF_8)
```

Parameter

path - the file

Return

the List of the lines as a List; whether the List is modifiable or not is implementation dependent and therefore not specified

Throws:

IOException - if an I/O error occurs reading from the file or a malformed or unmappable byte sequence is read

SecurityException - In the case of the default provider, and a security manager is installed, the `checkRead` method is invoked to check read access to the file.

Since:

1.8

Macht etwas Riskantes,
was zur Laufzeit vielleicht
nicht funktioniert.

Wir möchten wissen, dass die
aufgerufene Methode riskant ist.

Wir schreiben Code, der das
Problem behandelt, falls es auftritt...

write

```
1 import java.io.IOException;
2 import java.nio.file.Files;
3 import java.nio.file.Path;
4 import java.util.List;
5 import java.util.Scanner;
6
7 public class TryCatchDemo {
8
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         System.out.println("Please enter file path: ");
12         String filePath = scanner.nextLine();
13
14         // We *try* to do some risky operations that may fail at runtime:
15         try {
16             System.out.format("Trying to read lines from file: %s (may fail)...%n", filePath);
17
18             // Risky: The file may not exist or we may not have permission to read it.
19             List<String> lines = Files.readAllLines(Path.of(filePath));
20
21             System.out.println("Successfully read the following lines:");
22             for (String line : lines) {
23                 System.out.println(line);
24             }
25             // If the above code fails at runtime, an IOException is thrown.
26             // We *catch* this exception to handle the error:
27         } catch (IOException e) {
28             System.out.println("Could not read from file due to: " + e.getClass().getName());
29             System.out.println("Message: " + e.getMessage());
30         }
31     }
32 }
33
```

STDIN

hello-world-uh-oh.txt

Output:

Please enter file path:
Trying to read lines from file: hello-world-uh-oh.txt (may fail)...
Could not read from file due to: java.nio.file.NoSuchFileException
Message: hello-world-uh-oh.txt

! **Funktioniert
im OneCompiler
nicht (mehr).**

„Riskante Methoden, die zur Laufzeit fehlschlagen können,
deklarieren mögliche Ausnahmen mittels throws
SomeKindOfException (wirft IrgendEineAusnahme)
innerhalb ihrer Methodendeklaration.“

Merle – aufmerksame Student:in im 1. Semester





„Und denken Sie daran: Sie müssen dem Compiler mit Hilfe von try/catch mitteilen, dass Sie sich des Risikos bewusst sind und dass Sie sich darum kümmern!“

„Wenn ich weiß, dass beim Aufruf einer bestimmten Methode eine Ausnahmesituation auftreten kann, kann ich mich darauf vorbereiten. Vielleicht kann ich mich sogar von dem Problem erholen, das die Exception verursacht hat.“

Gerit – aufgeweckte Student:in im 1. Semester



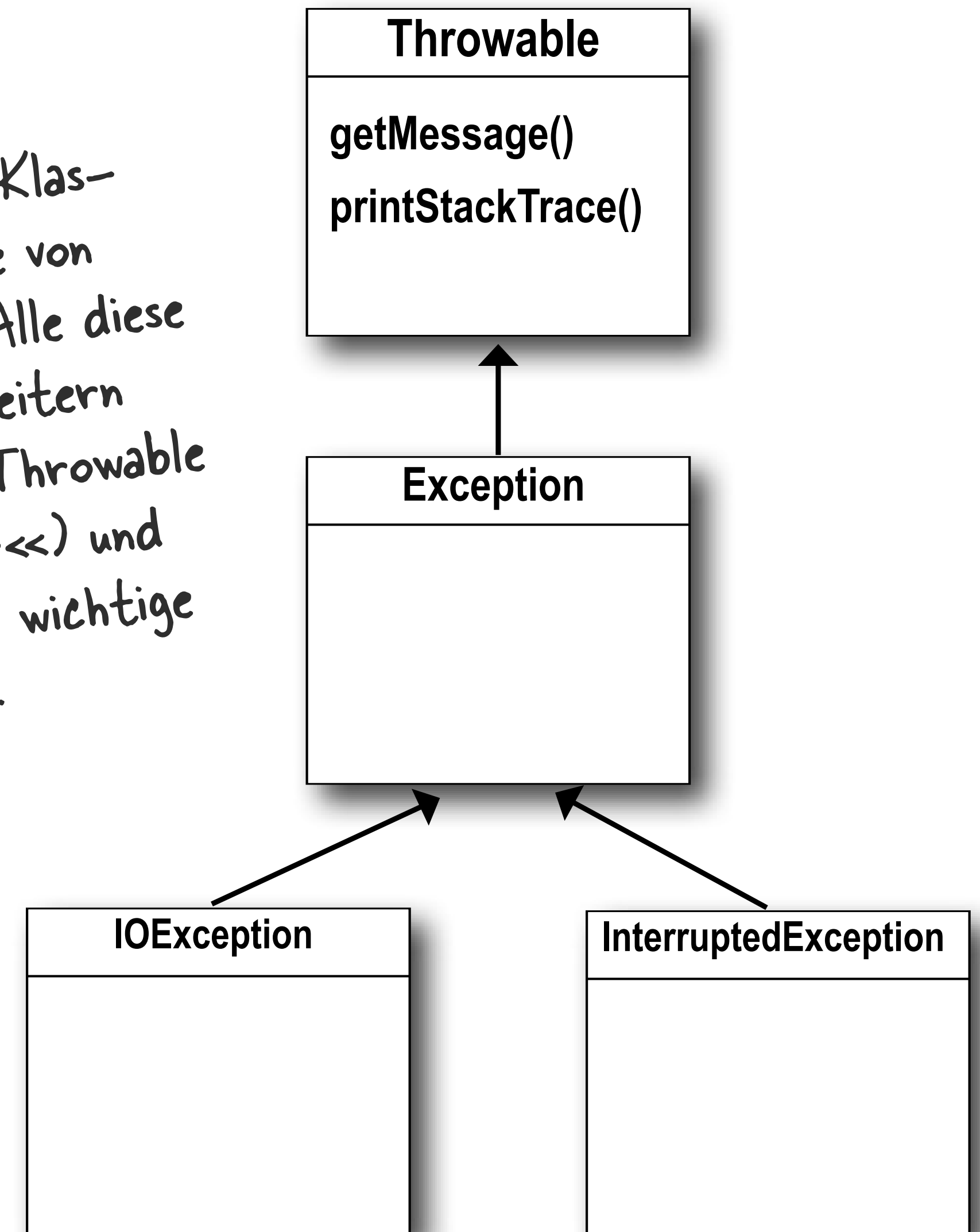
Exception Ist-Ein Object

🔥 Houston, wir haben eine Exception

- Aus dem vorherigen Inhalt zur Polymorphie wissen Sie, dass ein Objekt vom Typ `Exception` eine Instanz einer beliebigen Unterklasse von `Exception` sein kann.
- Da eine Ausnahme ein Objekt ist, ist auch das, was Sie „fangen“ (catch), ein `Object`.
- Deklaration des catch-Arguments mit dem Typ `Exception` und dem Namen `e` für die Referenzvariable (Argument):

```
try {  
    // tut was Riskantes  
} catch(Exception e)  
    // rettet / stellt wieder her  
}
```

Ein Teil der Klassenhierarchie von `Exception`. Alle diese Klassen erweitern die Klasse `Throwable` (>>werfbar<<) und erben zwei wichtige Methoden.



Jetzt Seite 429 wiederholen!



Sie haben dafür 3 Minuten Zeit.

! ? Quiz

🔥 Houston, wir haben eine **Exception**

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Eine Methode kann eine Exception auslösen, wenn zur Laufzeit ein Fehler auftritt.

Richtig!

Eine `Exception` ist eine von `Object`
losgelöste Kontroll- und Datenstruktur.

... ist immer ein Objekt des Typs `Exception`.

Um eine Ausnahme auszulösen (zu „werfen“), verwendet eine Methode das Schlüsselwort `throw`, gefolgt von einem neuen `Exception`-Objekt:

```
throw new OhNoItsMondayException();
```

Richtig!

Wenn Ihr Code eine solche Methode aufruft, die eine `Exception` auslösen könnte, müssen Sie dem Compiler bestätigen, dass Sie Vorsichtsmaßnahmen getroffen haben.

Richtig!

Um die mögliche Ausnahme eines Methodenaufrufs zu behandeln, packen Sie den Aufruf in einen `attempt/capture`-Block. Der Code zur Behandlung der Ausnahme steht im `capture`-Block.

`attempt/capture... capture-Block` → `try/catch... catch-Block`

 Welcher **eine** Gedanke zu
„ Houston, wir haben eine
Exception“ war für Sie am
wichtigsten?  Schreiben Sie ihn auf!

 Durchatmen und  Sortieren



Geprüfte und Ungeprüfte

Jetzt Seite 430 wiederholen!



Sie haben dafür 3 Minuten Zeit.

! ? Quiz

🧑 Geprüfte und Ungeprüfte

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Ausnahmen vom Typ `RuntimeException` sind für den Compiler ohne Bedeutung. Eine `RuntimeException` muss weder deklariert noch in einen `try/catch`-Block verpackt werden. Beides ist jedoch erlaubt.

Richtig!

Für den Compiler sind nur die vom Compiler explizit geprüften (checked) Exceptions von Bedeutung. Nur ungeprüfte (unchecked) RuntimeExceptions sind von der Überprüfung durch den Compiler ausgenommen; alle anderen Ausnahmen müssen von Ihnen in Ihrem Code bestätigt werden.

Richtig!

Methoden, die eine überprüfte Ausnahme werfen könnten, sollten dies mit der Deklaration `throws IrgendEineException` ankündigen.

sollten → müssen dies ankündigen

 Welcher **eine** Gedanke zu
„ Geprüfte und Ungeprüfte“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

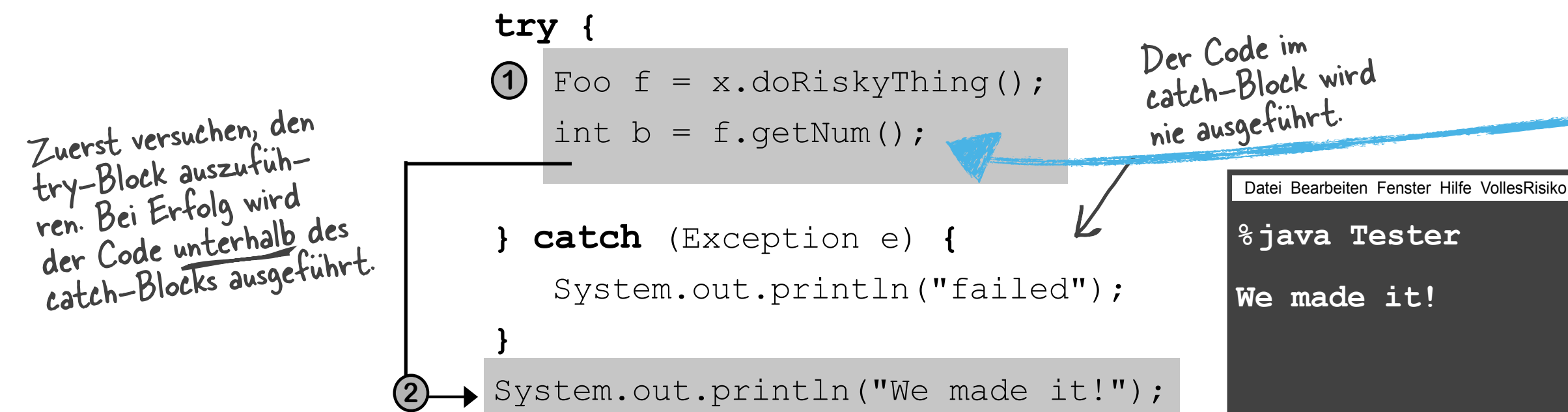
 Durchatmen und  Sortieren

Flusskontrolle

Aufruf einer riskanten Methode in try/catch

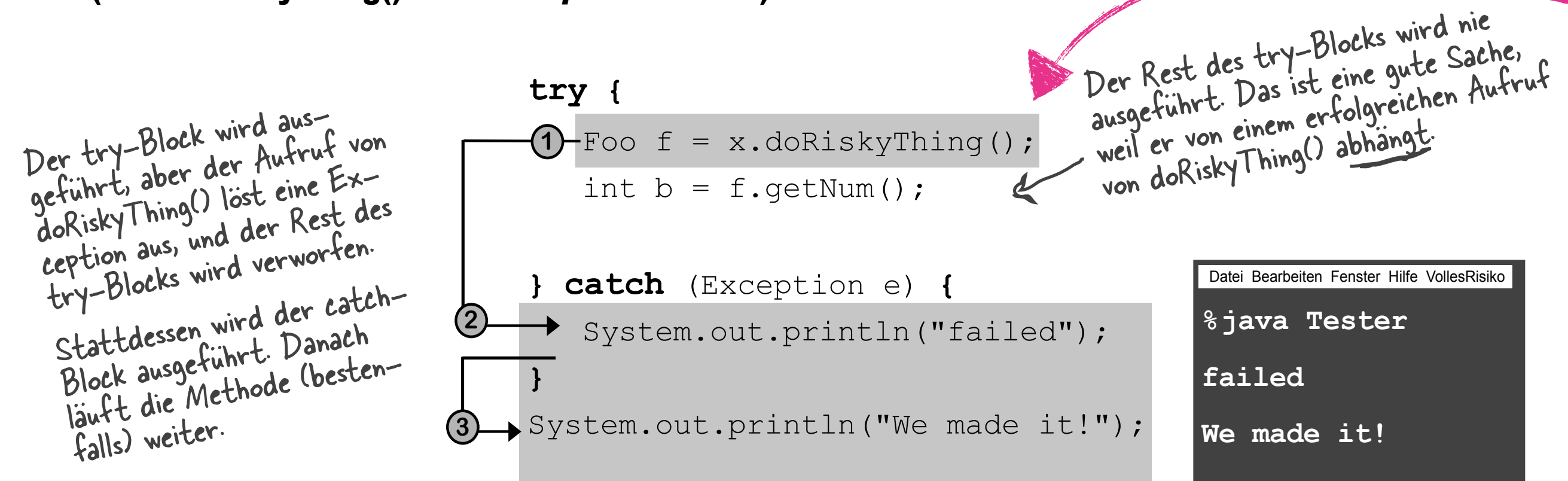
Flusskontrolle

Wenn das try erfolgreich ist
(doRiskyThing() löst keine Exception aus)



Entweder ist die riskante Methode erfolgreich und der try-Block wird zu Ende ausgeführt...

Wenn das try fehlschlägt
(weil doRiskyThing() eine Exception auslöst)



... oder sie wirft eine Exception an ihre aufrufende Methode zurück.

! ? Quiz

🔄 Flusskontrolle

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!

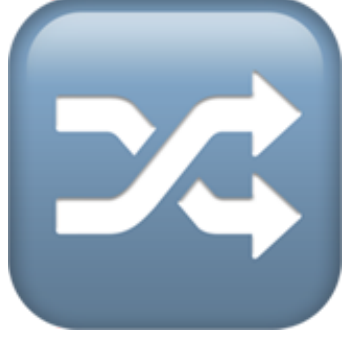
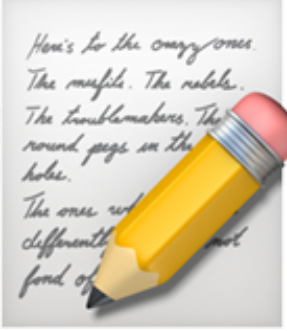


Zur Laufzeit wird zunächst versucht, den `try`-Block auszuführen. Im Erfolgsfall wird der Code nach dem `catch`-Block ausgeführt.

Richtig!

Zur Laufzeit wird zunächst die Ausführung des try-Blocks versucht. Im Falle eines Fehlers wird der catch-Block anstelle des Codes nach dem catch-Block ausgeführt.

Der Rest des try-Blocks wird nicht ausgeführt. Der Code wird nach dem catch-Block fortgesetzt.

 Welcher **eine** Gedanke zu
„ Flusskontrolle“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren



finally-Block



„Nein, nein, diese Code-Duplizierung
können wir uns dank des Exception-
Handlers try/catch/finally sparen...“

„Was ist mit Code, der auf jeden Fall ausgeführt
werden muss? Muss ich ihn sowohl in den
try- als auch in den catch-Block einfügen?“

Noel – neugierige Student:in im 1. Semester





„Sehr gute Frage! Mal schauen, was die offizielle Java-Dokumentation dazu sagt...“

„Moment mal, wenn der Code nach dem catch-Block sowieso ausgeführt wird, wozu brauche ich dann `finally`?“

Merle – aufmerksame Student:in im 1. Semester



„Der `finally`-Block wird immer ausgeführt, wenn der `try`-Block endet. Dies stellt sicher, dass der `finally`-Block auch bei unerwarteten Ausnahmen ausgeführt wird. Der `finally`-Block ist jedoch nicht nur für die Behandlung von Ausnahmen nützlich. **Er ermöglicht es der Programmierer:in, zu verhindern, dass der Aufräumcode versehentlich durch `return`, `continue` oder `break` umgangen wird.** Auch wenn keine Ausnahmen erwartet werden, ist das Einfügen von Aufräumcode in einen `finally`-Block eine gute Praxis.“



Frei übersetzt aus: Dev.java › Tutorials › Exceptions › Catching and Handling Exceptions

```
1 import java.io.BufferedReader;
2 import java.io.IOException;
3 import java.nio.file.Files;
4 import java.nio.file.Path;
5 import java.util.Scanner;
6
7 public class TryCatchFinallyDemo {
8
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         System.out.println("Please enter file path: ");
12         String filePath = scanner.nextLine();
13
14         // Spoiler Alert -> Serialisierung und Datei-I/O (Kap. 16)
15         BufferedReader reader = null;
16
17         try {
18             System.out.format("Trying to read lines from file: %s (may fail)...\n", filePath);
19             reader = Files.newBufferedReader(Path.of(filePath));
20
21             String line;
22             while ((line = reader.readLine()) != null) {
23                 System.out.println(line);
24             }
25         } catch (IOException e) {
26             // We use System.err instead of System.out to print the error message.
27             // System.err: Standard error stream (stderr) -> usually used for error messages.
28             System.err.println("Could not read from file due to: " + e.getClass().getName());
29             System.err.println("Message: " + e.getMessage());
30         } finally {
31             // A finally block contains the code that must be executed at all times,
32             // regardless of whether there has been an exception or not.
33             // Hint: The Try-with-resources Statement to the rescue (Kap. 16)!
34             // https://dev.java/learn/exceptions/catching-handling/#try-with-resources
35             if (reader != null) {
36                 // Please note that closing the file itself can trigger an IOException.
37                 try {
38                     System.out.println("Trying to close the reader (may fail)...");
39                     reader.close();
40                     System.out.println("Reader closed successfully.");
41                 } catch (IOException e) {
42                     System.err.println("Error closing the reader: " + e.getMessage());
43                 }
44             } else {
45                 System.err.println("Reader was not opened.");
46             }
47         }
48     }
49 }
50
```

STDIN

hello-world-uh-oh.txt

Output:

Please enter file path:
Trying to read lines from file: hello-world-uh-oh.txt (may fail)...

Could not read from file due to: java.nio.file.NoSuchFileException
Message: hello-world-uh-oh.txt
Reader was not opened.

! funktioniert
im OneCompiler
nicht (mehr).

! ? Quiz

🔧 **finally-Block**

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Einem try-Block müssen immer ein catch-Block und ein finally-Block folgen.

Einer von beiden oder beide.

Ein `finally`-Block wird immer ausgeführt.
Dabei ist es unerheblich, ob eine Ausnahme
geworfen wurde oder nicht.

Richtig!

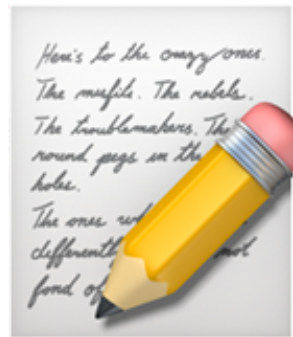


Welcher **eine** Gedanke zu



„finally-Block“ war für

Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren

Pair Programming

🎓 Geprüfte Exceptions

🤝 Pair Programming

- 🎯 Schreiben Sie ein Programm, das eine Benutzereingabe ohne Verwendung eines Scanners einliest und ausgibt.
- ✅ Erstellen Sie dazu eine Instanz von `BufferedReader`:
- ✅

```
new BufferedReader(  
    new InputStreamReader(  
        System.in))
```
- ✅ Verwenden Sie dessen Methode `readLine()`, um eine Zeile einzulesen.



Geprüfte Exceptions

Pair Programming

 **Schreiben Sie ein Programm, das eine Benutzereingabe ohne Verwendung eines Scanners einliest und ausgibt.**

 Erstellen Sie dazu eine Instanz von `BufferedReader`:

 `new BufferedReader(
 new InputStreamReader(
 System.in))`

 Verwenden Sie dessen Methode `readLine()`, um eine Zeile einzulesen.

 Diese Methode deklariert eine `IOException`, die zu den geprüften Ausnahmen gehört. Behandeln Sie diese entsprechend.

 Stellen Sie sicher, dass die Ressource in jedem Fall geschlossen wird.

 Am Ende sollten Sie wie gewohnt eine Benutzereingabe einlesen können – nur ohne die Verwendung der Klasse `Scanner`.


```
1 import java.io.BufferedReader;
2 import java.io.IOException;
3 import java.io.InputStreamReader;
4
5 public class CheckedExceptions {
6     public static void main(String[] args) {
7         BufferedReader reader = null;
8         try {
9             reader = new BufferedReader(new InputStreamReader(System.in));
10            System.out.println("Please enter something:");
11            String input = reader.readLine();
12            System.out.println("You entered: " + input);
13        } catch (IOException e) {
14            System.err.println("An error occurred while reading input: " + e.getMessage());
15        } finally {
16            if (reader != null) {
17                try {
18                    reader.close();
19                } catch (IOException e) {
20                    System.err.println("Failed to close the reader: " + e.getMessage());
21                }
22            }
23        }
24    }
25 }
26
```

STDIN

Hello, world!

Output:

Please enter something:
You entered: Hello, world!

 Welcher **eine** Gedanke zu
„ Geprüfte Exceptions“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Exception-Handling

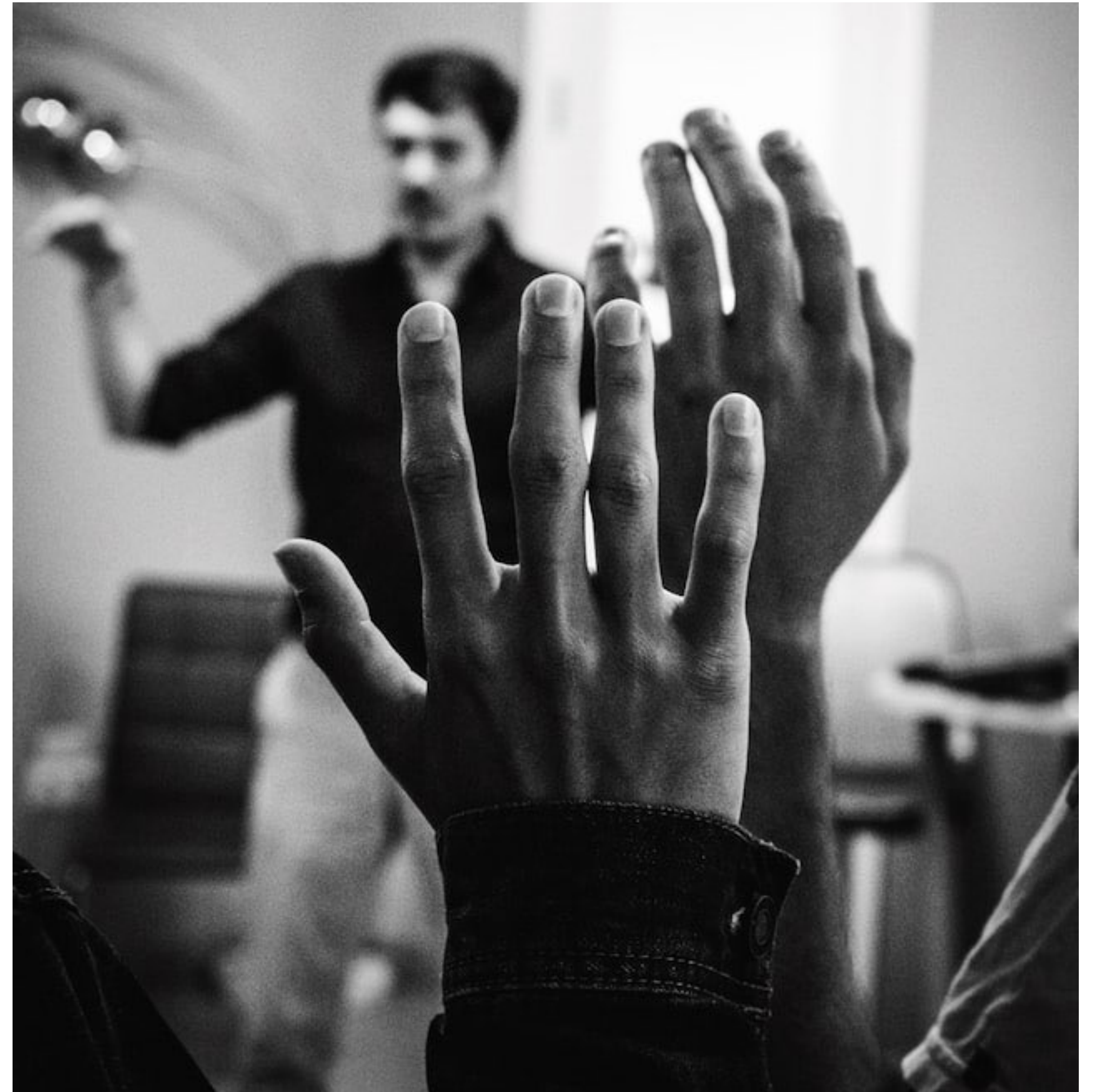
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Exception-Handling**?
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Exception-Handling



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

